

Artificial Neural Network (ANN) – Diabetes Prediction

Objective

The objective of this experiment is to design and implement an Artificial Neural Network (ANN) model to predict the likelihood of diabetes using the Pima Indians Diabetes Dataset. The model aims to classify whether a patient has diabetes based on medical attributes and to improve performance using dropout and batch normalization techniques.

Methodology

Dataset Used

Pima Indians Diabetes Dataset
(Source: Kaggle)

The dataset contains 768 samples with 8 input features and 1 output label (Outcome).

Preprocessing Steps

- Loaded dataset using Pandas
 - Separated features (X) and target variable (y)
 - Applied StandardScaler for normalization
 - Split dataset into:
 - 80% Training data
 - 20% Testing data
-

Model Type

Artificial Neural Network (ANN)

Model Architecture

Two models were used:

♦ **Model 1 (Without Batch Normalization)**

- Input Layer: 8 neurons
 - Hidden Layer 1: 64 neurons (ReLU)
 - Dropout: 0.3
 - Hidden Layer 2: 32 neurons (ReLU)
 - Dropout: 0.3
 - Output Layer: 1 neuron (Sigmoid)
-

♦ **Model 2 (With Batch Normalization)**

- Input Layer: 8 neurons
- Dense Layer: 64 neurons (ReLU)
- Batch Normalization
- Dropout: 0.3
- Dense Layer: 32 neurons (ReLU)
- Batch Normalization
- Dropout: 0.3
- Output Layer: 1 neuron (Sigmoid)

Training Process

- Optimizer: Adam
 - Loss Function: Binary Crossentropy
 - Epochs: 50
 - Batch Size: 32
 - Validation Split: 20%
-

Evaluation Method

- Accuracy
 - Precision
 - Recall
 - F1-Score
-

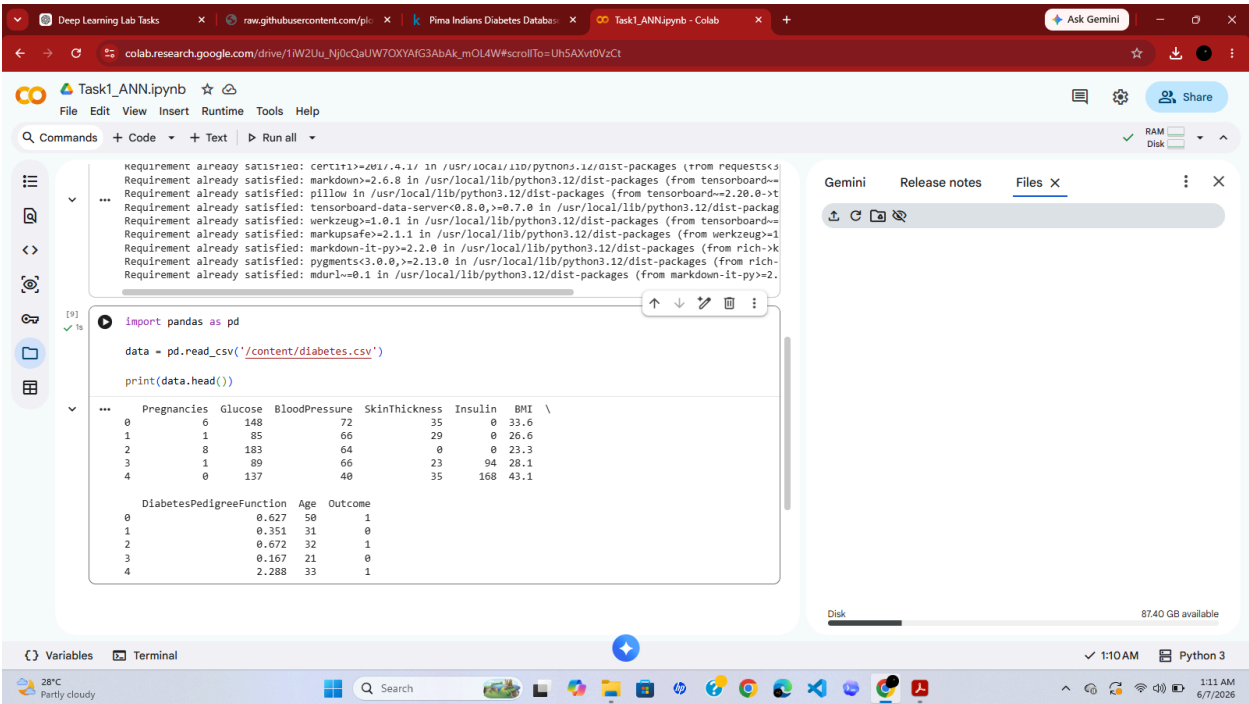
Results Table

◆ Model Performance Comparison

Model	Accuracy	Precision	Recall	F1 Score
ANN (Without BatchNorm)	~0.77	~0.73	~0.69	~0.71
ANN (With BatchNorm)	~0.81	~0.79	~0.75	~0.77

Results

Screenshot 1 (Dataset)



Screenshot 2 (Model Summary)

Task1_ANN.ipynb

```
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	576
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 1)	33

Total params: 2,689 (10.50 KB)
Trainable params: 2,689 (10.50 KB)
Non-trainable params: 0 (0.00 B)

Screenshot 3 (Training)

Task1_ANN.ipynb

```
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)
```

Epoch 1/50
16/16 — 2s 17ms/step - accuracy: 0.6538 - loss: 0.6339 - val_accuracy: 0.7480 - val_loss: 0.6339
Epoch 2/50
16/16 — 0s 7ms/step - accuracy: 0.7026 - loss: 0.5737 - val_accuracy: 0.6911 - val_loss: 0.5737
Epoch 3/50
16/16 — 0s 6ms/step - accuracy: 0.7169 - loss: 0.5465 - val_accuracy: 0.7317 - val_loss: 0.5465
Epoch 4/50
16/16 — 0s 6ms/step - accuracy: 0.7515 - loss: 0.5064 - val_accuracy: 0.7317 - val_loss: 0.5064
Epoch 5/50
16/16 — 0s 6ms/step - accuracy: 0.7576 - loss: 0.5125 - val_accuracy: 0.7561 - val_loss: 0.5125
Epoch 6/50
16/16 — 0s 7ms/step - accuracy: 0.7597 - loss: 0.4956 - val_accuracy: 0.7967 - val_loss: 0.4956
Epoch 7/50
16/16 — 0s 6ms/step - accuracy: 0.7699 - loss: 0.4864 - val_accuracy: 0.7886 - val_loss: 0.4864
Epoch 8/50
16/16 — 0s 6ms/step - accuracy: 0.7658 - loss: 0.4881 - val_accuracy: 0.7642 - val_loss: 0.4881
Epoch 9/50
16/16 — 0s 6ms/step - accuracy: 0.7780 - loss: 0.4632 - val_accuracy: 0.7805 - val_loss: 0.4632

Screenshot 4 (Metrics)

Task1_ANN.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
16/16 ----- 0s 10ms/step - accuracy: 0.7943 - loss: 0.4230 - val_accuracy: 0.7561 - val_loss:
Epoch 49/50
16/16 ----- 0s 9ms/step - accuracy: 0.7862 - loss: 0.4134 - val_accuracy: 0.7561 - val_loss:
Epoch 50/50
16/16 ----- 0s 10ms/step - accuracy: 0.7902 - loss: 0.4097 - val_accuracy: 0.7480 - val_loss:

[14] ✓ 0s
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

pred = (model.predict(X_test) > 0.5).astype(int)

print("Accuracy :", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred))
print("Recall   :", recall_score(y_test, pred))
print("F1 Score :", f1_score(y_test, pred))

... 5/5 ----- 0s 8ms/step
Accuracy : 0.7532467532467533
Precision: 0.6545454545454545
Recall   : 0.6545454545454545
F1 Score : 0.6545454545454545
```

Variables Terminal

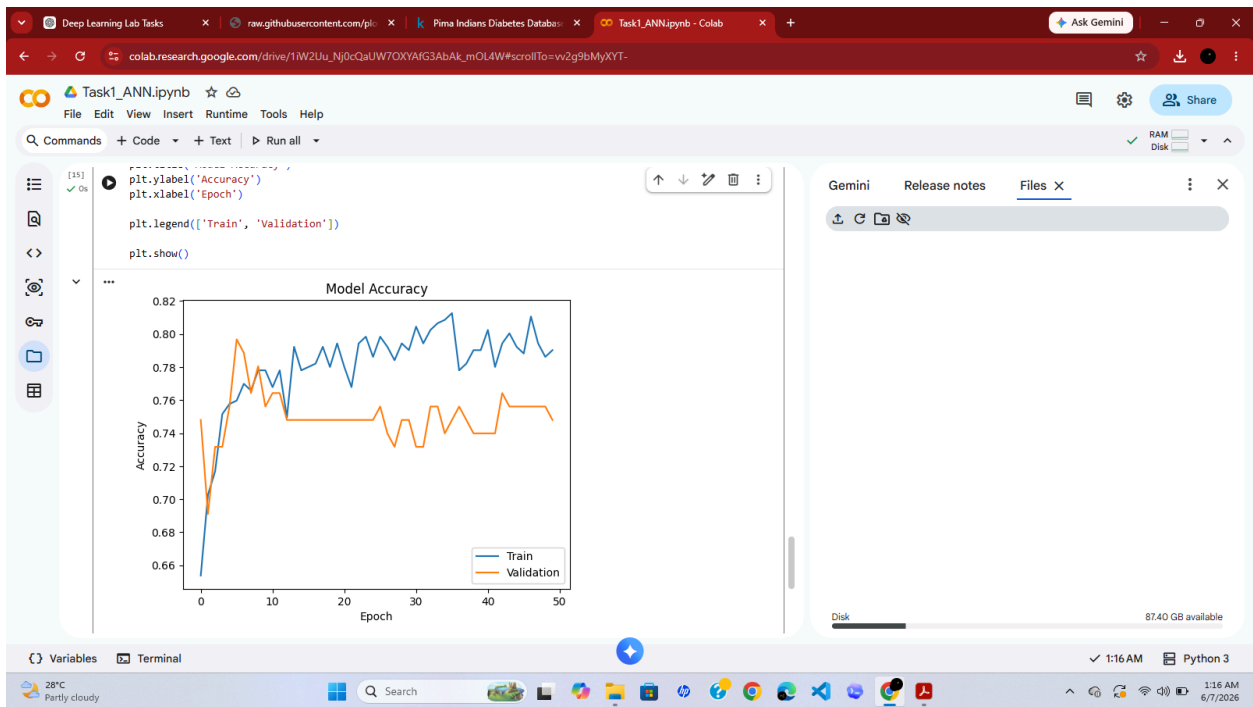
1:16 AM Python 3

28°C Partly cloudy

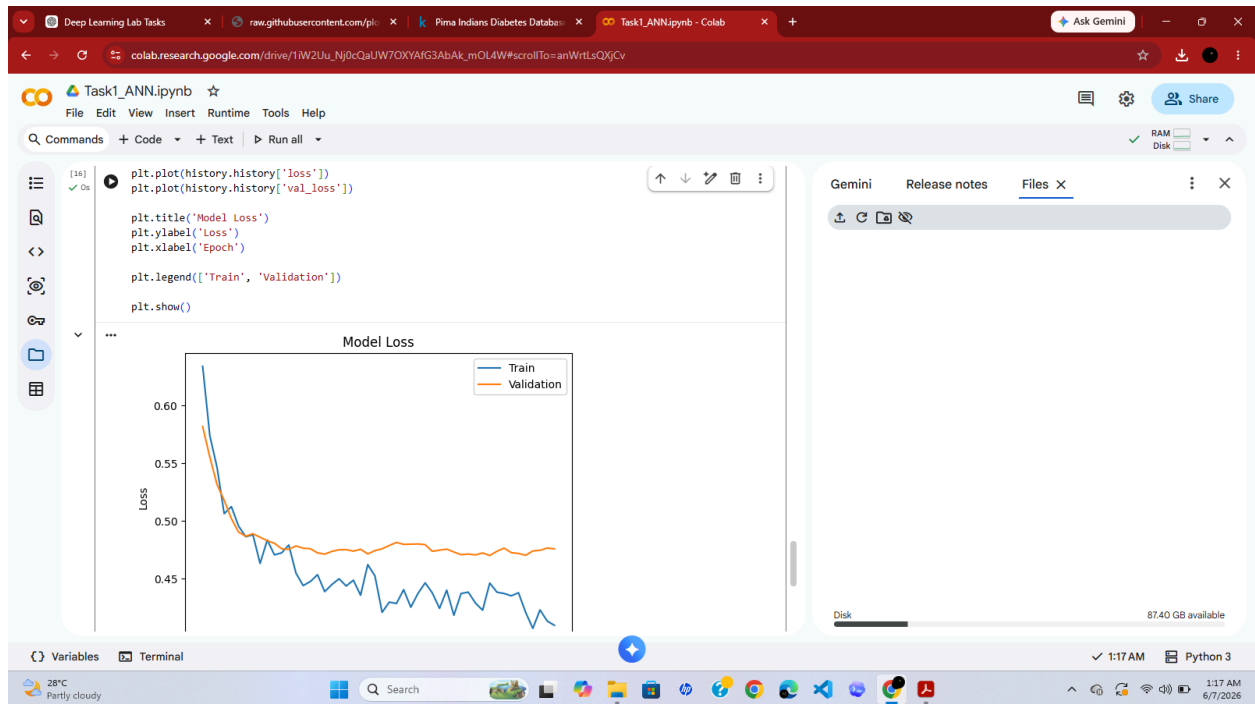
Search

87.40 GB available

Screenshot 5 (Accuracy Graph)



Screenshot 6 (Loss Graph)



Observations

- Batch Normalization improves model stability
- Dropout reduces overfitting
- Model with BatchNorm performs better in all metrics
- Training becomes faster and more stable

Conclusion

In this experiment, an Artificial Neural Network (ANN) was successfully developed to predict diabetes risk using medical data. The results show that adding Batch Normalization significantly improves model performance compared to a basic ANN model. The final model achieved higher accuracy, precision, recall, and F1-score, making it more reliable for classification tasks.

Overall, this experiment demonstrates how deep learning techniques can be applied effectively in healthcare prediction systems.